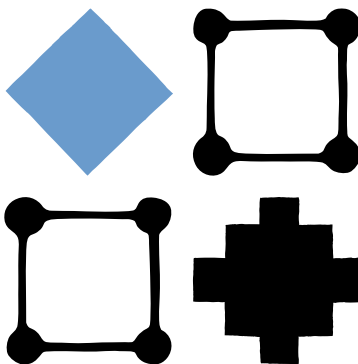


think 工具：让 Claude 在复杂工具使用场景中停下来思考

The "think" tool: Enabling Claude to stop and think in complex tool use situations

Anthropic · 2025-03-20 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



一种提升 Claude 复杂问题解决能力的新工具

- 扩展思考功能更新

2025 年 12 月 15 日

扩展思考能力自首次发布以来已有提升，因此，在大多数情况下，我们建议使用该功能，而非专门的 think 工具。扩展思考同样能为 Claude 提供推理复杂问题的空间，同时具备更好的集成方式和性能。实现细节请参阅扩展思考文档。

在持续增强 Claude 解决复杂问题的能力时，我们发现了一种特别有效的方法：“think”工具。它为复杂任务中的结构化思考提供了专门的空间。

这个简单而强大的技巧，与 Claude 新推出的“扩展思考”能力并不相同，后文会解释两者的区别（扩展思考的实现细节见此处）。它显著改善了 Claude 在 Agent 工作中的工具使用能力，包括遵循规则、作出一致的决策和处理多步骤问题，而实现开销很小。

本文将探讨如何在不同应用中实现“think”工具，并根据经过验证的基准测试结果，为开发者提供实用指导。

什么是“think”工具？

通过“think”工具，我们让 Claude 能够在得出最终答案的过程中，加入一个额外的思考步骤，并为其配备专用空间。

虽然听起来与扩展思考类似，但这是不同的概念。扩展思考关注的是 Claude 开始生成回答之前所做的事情。在扩展思考中，Claude 会在采取行动前深入考虑并迭代自己的计划。“think”工具则让 Claude 在已经开始生成回答之后，增加一个停下来思考的步骤，判断自己是否已经掌握了继续推进所需的全部信息。当执行很长的工具调用链，或与用户进行漫长的多步骤对话时，这尤其有帮助。

因此，“think”工具更适合这样的场景：Claude 仅凭用户查询还无法获得形成回答所需的全部信息，并且需要处理外部信息，例如工具调用结果中的信息。Claude 使用“think”工具时进行的推理，不像扩展思考所能提供的那样全面，而是更聚焦于模型发现的新信息。

对于不依赖先后顺序的工具调用、直接的指令遵循等较简单的工具使用场景，我们建议使用扩展思考。在编程、数学、物理等不需要 Claude 调用工具的用例中，扩展思考也很有用。“think”工具则更适合以下情况：Claude 需要调用复杂工具，在长工具调用链中仔细分析工具输出，处理有详细指导规则的复杂制度环境，或连续作出每一步都依赖此前步骤、而且出错代价很高的决策。

下面是一个使用标准工具规格格式的实现示例，来自 τ -Bench：

```
{
  "name": "think",
  "description": "Use the tool to think about something. It will not obtain new information or change the database, but just append the thought to the log. Use it when complex reasoning or some cache memory is needed.",
  "input_schema": {
    "type": "object",
    "properties": {
      "thought": {
        "type": "string",
        "description": "A thought to think about."
      }
    }
  },
  "required": ["thought"]
}
```

在 τ -Bench 上的表现

我们使用 τ -bench（tau-bench）评估了“think”工具。这是一项综合性基准测试，旨在测试模型在真实客服场景中的工具使用能力；“think”工具是该评测标准环境的一部分。

τ -bench 评估 Claude 的以下能力：

- 与模拟用户开展贴近真实情况的对话
- 始终如一地遵循复杂的客服 Agent 规则与指导要求
- 使用多种工具访问和操作环境中的数据库

τ -bench 使用的主要评测指标是 pass^k ，它衡量的是：对于给定任务， k 次独立尝试全部成功的概率，再对所有任务取平均。其他大语言模型评测中常见的 $\text{pass}@k$ 指标，衡量的是 k 次尝试中是否至少有一次成功；与之不同， pass^k 评估的是一致性与可靠性。这些品质对客服应用至关重要，因为这类应用必须始终遵守规则。

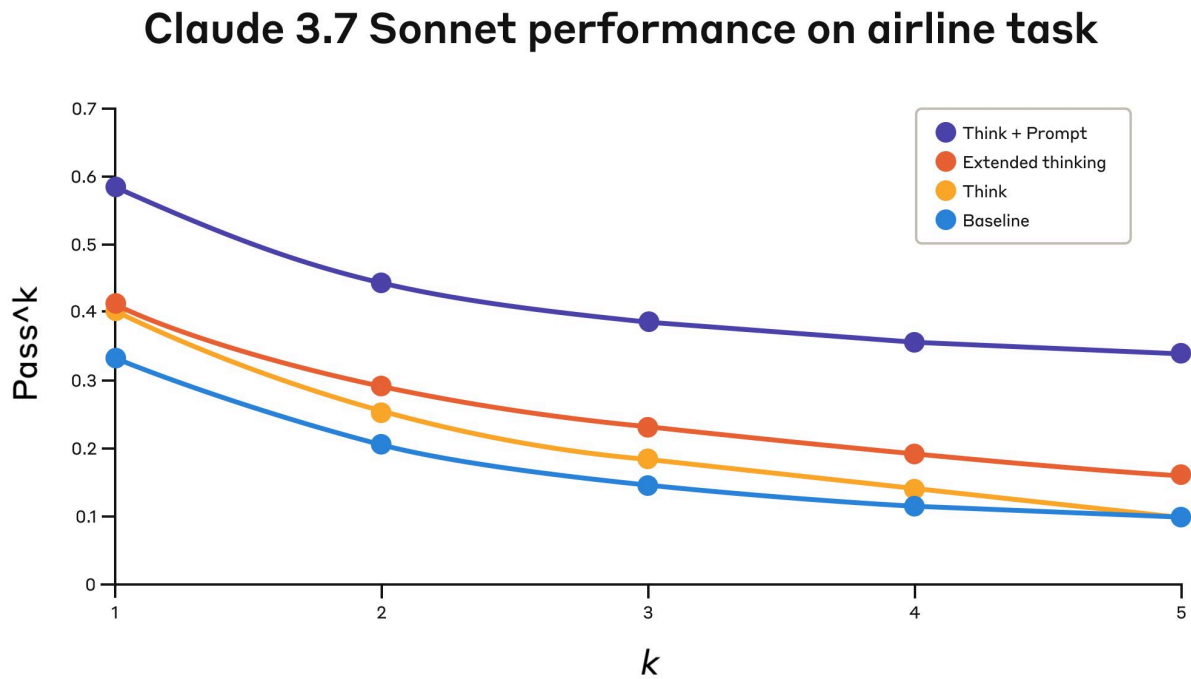
性能分析

我们的评测比较了几种不同配置：

- 1. 基线配置（不使用“think”工具，也不开启扩展思考模式）
- 2. 仅使用扩展思考模式
- 3. 仅使用“think”工具
- 4. “think”工具搭配优化后的提示词（针对航空业务场景）

结果表明，当 Claude 3.7 有效使用“think”工具时，在该基准测试的“航空”和“零售”客服场景中都取得了显著提升：

- **航空业务场景：**“think”工具搭配优化后的提示词，在 pass[^]1 指标上取得了 0.570 的成绩，而基线只有 0.370，相对提升了 54%；
- **零售业务场景：**仅使用“think”工具就达到了 0.812，基线则为 0.783。



Claude 3.7 Sonnet's performance on the "airline" domain of the Tau-Bench eval under four different configurations.

Claude 3.7 Sonnet 在 Tau-Bench 评测“航空”业务场景中的表现

Configuration	k=1	k=2	k=3	k=4	k=5
"Think" + Prompt	0.584	0.444	0.384	0.356	0.340
"Think"	0.404	0.254	0.186	0.140	0.100
Extended thinking	0.412	0.290	0.232	0.192	0.160
Baseline	0.332	0.206	0.148	0.116	0.100

四种不同配置的评测结果。分数以比例表示。

在航空业务场景中，将“think”工具与优化后的提示词搭配使用，取得了最佳表现。这段提示词通过示例展示了分析客户请求时应采用的推理方式。下面是优化后提示词的一个示例：

```
## Using the think tool

Before taking any action or responding to the user after receiving tool results, use the think tool as a scratchpad to:
- List the specific rules that apply to the current request
- Check if all required information is collected
- Verify that the planned action complies with all policies
- Iterate over tool results for correctness

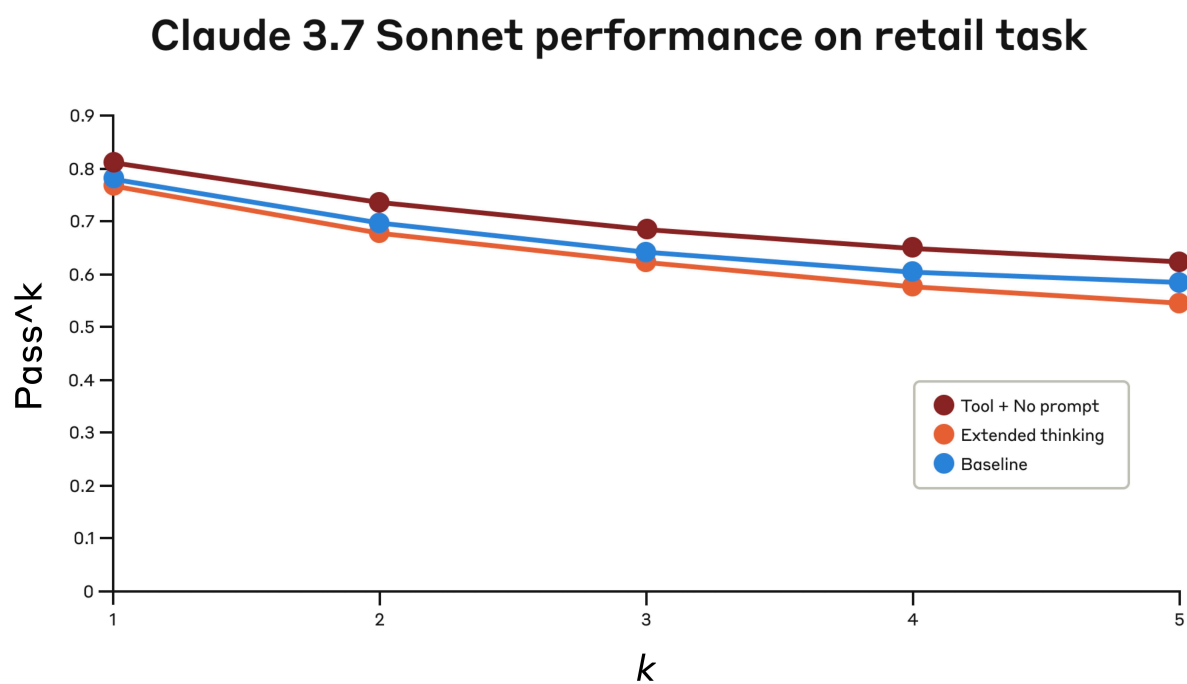
Here are some examples of what to iterate over inside the think tool:
<think_tool_example_1>
User wants to cancel flight ABC123
- Need to verify: user ID, reservation ID, reason
- Check cancellation rules:
  * Is it within 24h of booking?
  * If not, check ticket class and insurance
- Verify no segments flown or are in the past
- Plan: collect missing info, verify rules, get confirmation
</think_tool_example_1>

<think_tool_example_2>
User wants to book 3 tickets to NYC with 2 checked bags each
- Need user ID to check:
  * Membership tier for baggage allowance
  * Which payments methods exist in profile
- Baggage calculation:
  * Economy class × 3 passengers
  * If regular member: 1 free bag each → 3 extra bags = $150
  * If silver member: 2 free bags each → 0 extra bags = $0
  * If gold member: 3 free bags each → 0 extra bags = $0
- Payment rules to verify:
  * Max 1 travel certificate, 1 credit card, 3 gift cards
  * All payment methods must be in profile
  * Travel certificate remainder goes to waste
- Plan:
1. Get user ID
2. Verify membership level for bag fees
3. Check which payment methods in profile and if their combination is allowed
4. Calculate total: ticket price + any bag fees
5. Get explicit confirmation for booking
</think_tool_example_2>
```

特别有意思的是不同方法之间的比较。“think”工具搭配优化后的提示词，明显优于扩展思考模式；扩展思考模式的表现则与不附加专门提示词的“think”工具相近。仅使用“think”工具而不添加专门提示词，也能超越基线，但仍不及优化后的方法。

“think”工具与优化提示词的组合，以明显优势取得了最强表现。这可能是因为基准测试中的航空业务规则十分复杂，因此模型在获得如何“思考”的示例后，受益最大。

在零售业务场景中，我们也测试了多种配置，以了解各个方法的具体影响。



Performance of Claude 3.7 Sonnet on the "retail" domain of the Tau-Bench eval under three different configurations.

Claude 3.7 Sonnet 在 Tau-Bench 评测“零售”业务场景中的表现

Configuration	k=1	k=2	k=3	k=4	k=5
"Think" + no prompt	0.812	0.735	0.685	0.650	0.626
Extended thinking	0.770	0.681	0.623	0.581	0.548
Baseline	0.783	0.695	0.643	0.607	0.583

三种不同配置的评测结果。分数以比例表示。

即使没有额外提示词，“think”工具也取得了最高的 pass¹ 分数 0.812。与航空业务场景相比，零售业务规则明显更容易处理，Claude 只要拥有思考空间，不需要进一步指导，就能取得提升。

τ-Bench 分析中的关键发现

我们的详细分析揭示了几种规律，可以帮助你有效实现“think”工具：

- 1. 在困难场景中，提示词的影响很大。仅提供“think”工具，可能就会带来一些性能提升，但在困难场景中，将其与优化提示词搭配使用，结果会显著更好。对于较简单的场景，只需能够使用“think”就可能受益。
- 2. 各次尝试的一致性得到改善。使用“think”带来的提升，在 pass^k 指标的 k 增大到 5 时仍然保持，这说明该工具帮助 Claude 更有效地处理了边界情况和异常场景。

在 SWE-Bench 上的表现

评估 Claude 3.7 Sonnet 时，我们也在 SWE-bench 配置中加入了一个类似的“think”工具，为取得当时最先进的 0.623 分成绩作出了贡献。调整后的“think”工具定义如下：

```
{
  "name": "think",
  "description": "Use the tool to think about something. It will not obtain new information or make any changes to the repository, but just log the thought. Use it when complex reasoning or brainstorming is needed. For example, if you explore the repo and discover the source of a bug, call this tool to brainstorm several unique ways of fixing the bug, and assess which change(s) are likely to be simplest and most effective. Alternatively, if you receive some test results, call this tool to brainstorm ways to fix the failing tests.",
  "input_schema": {
    "type": "object",
    "properties": {
      "thought": {
        "type": "string",
        "description": "Your thoughts."
      }
    },
    "required": ["thought"]
  }
}
```

我们的实验中，使用“think”工具的样本数为 $n=30$ ，不使用该工具的样本数为 $n=144$ 。实验显示，单独加入这个工具的效果，是使性能平均提升 1.6%（Welch t 检验： $t(38.89) = 6.71$, $p < .001$, $d = 1.47$ ）。

何时使用“think”工具

根据这些评测结果，我们确定了 Claude 使用“think”工具时受益最大的几种具体场景：

1. **分析工具输出。** Claude 需要在行动前仔细处理此前工具调用的输出，而且可能需要回头调整方法；
2. **规则繁多的环境。** Claude 需要遵循详细指导要求，并核实是否符合规则；以及
3. **序贯决策。** 每个动作都建立在之前动作的基础上，且出错代价很高，这在多步骤场景中很常见。

实现最佳实践

为充分发挥 Claude 使用“think”工具的效果，我们根据 τ -bench 实验，推荐以下实现做法。

1. 使用领域专属示例，有针对性地编写提示词

最有效的方法，是明确说明何时以及如何使用“think”工具，例如我们在 τ -bench 航空业务场景中采用的提示词。提供针对具体用例的示例，可以显著提高模型使用“think”工具的有效性。这些示例应说明：

- 推理过程预期达到的详细程度；
- 如何将复杂指令拆解为可执行步骤；
- 处理常见场景的决策树；以及
- 如何检查是否已经收集到全部必要信息。

2. 将复杂指导放入系统提示词

我们发现，当关于“think”工具的说明较长或较复杂时，将其放在系统提示词中，比放在工具描述本身更有效。这种做法提供了更广泛的上下文，帮助模型把思考过程更好地融入整体行为。

何时不应使用“think”工具

虽然“think”工具能带来显著提升，但它并不适用于所有工具使用场景，而且会增加提示词长度与输出 token。具体来说，我们发现，在以下用例中，“think”工具没有带来任何改善：

1. **不依赖先后顺序的工具调用。**如果 Claude 只需进行一次工具调用，或多次并行调用就能完成任务，那么加入“think”不太可能带来改善。
2. **简单的指令遵循。**当 Claude 需要遵守的约束不多，而默认行为已经足够好时，额外使用“think”来思考不太可能带来收益。

开始使用

将“think”工具加入 Claude 应用很简单，只需几个步骤，就可能带来有意义的改善：

1. **在 Agent 工具使用场景中测试。**从具有挑战性的用例入手，例如 Claude 目前在长工具调用链中难以遵守业务规则或完成复杂推理的情况。
2. **添加工具定义。**实现一个针对你的领域定制的“think”工具。它只需少量代码，却能支持更有结构的推理。同时，可以考虑在系统提示词中加入何时以及如何使用该工具的说明，并提供与你的领域相关的示例。
3. **监测并改进。**观察 Claude 在实际运行中如何使用该工具，并调整提示词，鼓励更有效的思考模式。

最有利的一点是，从实际性能结果来看，加入这个工具几乎没有负面影响。除非 Claude 决定使用它，否则它不会改变外部行为，也不会干扰现有工具或工作流。

结语

我们的研究表明，对于需要遵守业务规则、并在长工具调用链中进行推理的复杂任务，“think”工具可以显著提升 Claude 3.7 Sonnet 的性能¹。“think”并不是适合所有情况的通用解法，但在恰当的用例中，它能以很低的实现复杂度带来显著收益。

我们期待看到你如何使用“think”工具，与 Claude 一起构建能力更强、更可靠、更透明的 AI 系统。

1. 虽然 τ -Bench 的结果聚焦于“think”工具对 Claude 3.7 Sonnet 的改进，但我们的实验表明，Claude 3.5 Sonnet (New) 采用与 3.7 Sonnet 相同的配置，也能取得性能提升，这说明此项改进同样可以推广到其他 Claude 模型。